

# Implement Client Script & UI Policy (Incident)

## Phase 1: Ideation

| Item          | Details                                                                            |
|---------------|------------------------------------------------------------------------------------|
| Project Title | Implement Client Script & UI Policy (Incident)                                     |
| Programme     | SkillWallet – Vetri Thiran Payirchi                                                |
| Platform      | ServiceNow                                                                         |
| Target Table  | Incident [incident]                                                                |
| Key Skills    | Incident Management, UI Policy, UI Policy Actions, Client Scripts, Form Validation |
| Team ID       | SWTID-2026-7176                                                                    |

## 1. Introduction to the Ideation Phase

The Ideation Phase establishes why the project is needed and what will be built. It captures the problem faced by users of the Incident form, builds empathy with the people affected, generates candidate ideas, and prioritises them to arrive at a focused solution. This phase uses three activities:

1. Define the problem statement.
2. Build an empathy map of the primary users.
3. Brainstorm, group and prioritise ideas.

## 2. Define the Problem Statement

### 2.1 Background

Incident records drive triage, routing, SLA tracking and resolution. When data entry depends only on user awareness and manual checks, records can be submitted with missing, inconsistent or incorrect values. This weakens reporting accuracy, SLA compliance and overall service quality.

### 2.2 Customer Problem Statement

| Template Element    | Statement                                                                                                                                                                              |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| I am                | A service desk agent or incident manager working on the ServiceNow Incident form.                                                                                                      |
| I'm trying to       | Log and route incidents quickly, with complete and correct information, especially for high-impact incidents.                                                                          |
| But                 | Required fields such as Assignment group and Assigned to can be skipped, Urgency can be set inconsistently with Impact, and State can be changed in bulk from the list without review. |
| Because             | The form relies on manual checks and does not enforce business rules at the user interface level.                                                                                      |
| Which makes me feel | Uncertain about data quality, worried about missed SLAs and unreliable reports, and burdened with rework.                                                                              |

### 2.3 Problem Summary

- **Incomplete data:** High-impact incidents can be saved with no assignment group or assignee.

- **Inconsistent values:** Urgency may not reflect a High Impact selection.
- **Uncontrolled edits:** Users can change State directly in list view and bypass the form.
- **Business impact:** Bad data reduces reporting accuracy and delays triage and resolution.

### 3. Empathy Map Canvas

The empathy map is built around the primary user: the service desk agent who creates and updates Incident records.

| Quadrant | Observations                                                                                                                                           |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Says     | "I need to log this quickly." "Which group should own this?" "Why was this incident saved without an owner?"                                           |
| Thinks   | Wants every high-impact incident to reach the right team without delay; worries about SLA breaches and being blamed for incomplete records.            |
| Does     | Fills the form under time pressure, changes Impact and Urgency, edits State from the list view to save time, and re-opens records to fix missing data. |
| Feels    | Rushed, uncertain about mandatory information, frustrated by rework, and anxious about escalations.                                                    |
| Pains    | Missing assignment details, mismatched Impact and Urgency, accidental state changes from list edits, and inconsistent records across the team.         |
| Gains    | Clear guidance on required fields, automatic and consistent Urgency, protection from accidental changes, and reliable reports.                         |

Secondary stakeholders: the ServiceNow administrator, who wants maintainable and lightweight configuration, and the incident manager, who needs accurate reporting and SLA data.

### 4. Brainstorming and Idea Listing

Ideas were listed freely before any filtering. Each idea addresses a part of the problem statement.

| ID | Idea                                                  | How It Helps                                        | Mechanism                |
|----|-------------------------------------------------------|-----------------------------------------------------|--------------------------|
| I1 | Make Assignment group mandatory when Impact is High   | Ensures ownership is defined for critical incidents | UI Policy + Action       |
| I2 | Make Urgency read-only when Impact is High            | Prevents mismatched Impact and Urgency              | UI Policy Action         |
| I3 | Automatically set Urgency to High when Impact is High | Removes manual effort and inconsistency             | onChange Client Script   |
| I4 | Block save if Assigned to is empty for High Impact    | Stops incomplete high-impact records                | onSubmit Client Script   |
| I5 | Block State edits in list view                        | Forces changes through the reviewed form            | onCellEdit Client Script |
| I6 | Reverse policy effects when Impact is no longer High  | Keeps the form flexible for normal incidents        | Reverse if false         |
| I7 | Server-side Business Rule for the same checks         | Enforces rules for API and import paths too         | Business Rule            |

| ID | Idea                                     | How It Helps                         | Mechanism      |
|----|------------------------------------------|--------------------------------------|----------------|
| I8 | Data Policy for mandatory fields         | Enforces rules on all entry channels | Data Policy    |
| I9 | Training and manual checklist for agents | Raises awareness of required data    | Process change |

## 4.1 Idea Grouping

| Group                                 | Ideas      |
|---------------------------------------|------------|
| Field behaviour on the form           | I1, I2, I6 |
| Automation and validation on the form | I3, I4     |
| List-view control                     | I5         |
| Server-side and process controls      | I7, I8, I9 |

## 5. Idea Prioritization

Each idea was rated from 1 (low) to 5 (high) on impact and feasibility, and effort was rated from 1 (low effort) to 5 (high effort). The score is Impact + Feasibility – Effort.

| ID | Idea                                       | Impact | Feasibility | Effort | Score | Pick |
|----|--------------------------------------------|--------|-------------|--------|-------|------|
| I1 | Assignment group mandatory for High Impact | 5      | 5           | 1      | 9     | Yes  |
| I2 | Urgency read-only for High Impact          | 4      | 5           | 1      | 8     | Yes  |
| I3 | Auto-set Urgency on Impact change          | 5      | 5           | 2      | 8     | Yes  |
| I4 | Block save without Assigned to             | 5      | 5           | 2      | 8     | Yes  |
| I5 | Block State edits in list view             | 4      | 4           | 2      | 6     | Yes  |
| I6 | Reverse if false                           | 4      | 5           | 1      | 8     | Yes  |
| I7 | Server-side Business Rule                  | 4      | 4           | 4      | 4     | No   |
| I8 | Data Policy                                | 3      | 4           | 3      | 4     | No   |
| I9 | Training and checklist                     | 2      | 5           | 3      | 4     | No   |

I1 to I6 are selected because they are quick to configure, sit directly on the Incident form, and solve the stated problem at the user interface level. I7 to I9 are recorded as possible future enhancements.

## 6. Proposed Solution

The selected ideas form a solution made of one UI Policy, one UI Policy Action set and three Client Scripts, all on the Incident table.

| No. | Component                           | Type                                             | Behaviour                                                |
|-----|-------------------------------------|--------------------------------------------------|----------------------------------------------------------|
| 1   | High Impact Control                 | UI Policy (Impact is 1 - High, Reverse if false) | Makes Assignment group mandatory                         |
| 2   | Urgency action                      | UI Policy Action                                 | Makes Urgency read-only when Impact is High              |
| 3   | Auto set urgency for high impact    | Client Script: onChange (Impact)                 | Sets Urgency to 1 and shows an information message       |
| 4   | Prevent save if Assigned To missing | Client Script: onSubmit                          | Blocks save when Impact is High and Assigned to is empty |
| 5   | Prevent state change via list edit  | Client Script: onCellEdit (State)                | Alerts the user and cancels the list edit                |

## 6.1 Concept Snippet

The following onChange script shows the core idea: react to the Impact value and keep Urgency consistent.

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }
    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
```

## 7. Expected Outcomes

- High-impact incidents cannot be saved without an assignee.
- Urgency stays consistent with Impact and cannot be edited while Impact is High.
- State cannot be changed from the list view; changes go through the form.
- Fields return to normal behaviour when Impact is no longer High.
- Cleaner, more consistent Incident data, supporting accurate reports and SLA compliance.

## 8. Conclusion

The Ideation Phase turned a general data-quality concern into a clear problem statement, an understanding of user needs, and a prioritised set of ideas. The chosen approach uses ServiceNow UI Policies and Client Scripts to enforce validation at the point of entry. It is lightweight, quick to implement, and suited to a short-duration micro project. The next phases build on it with requirements, design, planning, development, documentation and demonstration.